

Nome: _____

- As questões que envolvem implementação devem ser resolvidas utilizando a linguagem C/C++.
- Utilize as folhas de resposta distribuídas junto com a prova para responder às questões.
- Em cada folha, escreva seu nome completo no canto superior esquerdo. Respostas em folhas sem identificação não serão corrigidas.
- As respostas podem ser escritas a lápis ou caneta. Certifique-se de que sua escrita está legível.
- Indique claramente o número da questão que está respondendo e, quando necessário, detalhe os passos utilizados para chegar à solução.
- Ao final da prova, as folhas de resposta devem ser grampeadas junto à prova.

Questão 1 (1.5 pontos) O custo computacional $T_i(n)$ de diversos algoritmos é mostrado abaixo:

- $T_1(n) = n! + 2^n$
- $T_2(n) = 3n^2 \log_2(n) + n^3$
- $T_3(n) = 5 \log_2(n) + (\log_2(n))^2$
- $T_4(n) = n^2 \log_2(n) + 100n^2$
- $T_5(n) = \log_2(2^n)$
- $T_6(n) = 10n + 5000 + 5n^{1.5}$
- $T_7(n, m) = m \log_2(n)$, onde $m \leq n^2$
- $T_8(n) = \log_2(n) + 3\sqrt{n}$

- (a) Qual a complexidade de pior caso de cada algoritmo?
- (b) Ordene essas funções da menor para a maior complexidade assintótica.

Questão 2 (2.0 pontos) A estrutura a seguir representa uma fila implementada com um array circular:

```
struct Queue {  
    int* values;    // array de inteiros  
    int size;       // quantidade atual de elementos na fila  
    int capacity;   // capacidade total do array  
    int head;       // índice do primeiro elemento  
    int tail;       // índice da próxima posição livre  
};
```

Sabendo que a fila armazena os elementos de forma circular, responda:

- (a) Implemente a função `int sequentialSearch(Queue* queue, int x)` que percorre os elementos da fila (respeitando a circularidade) e retorna 1 se `x` estiver presente, ou 0 caso contrário.
- (b) Implemente a função `int binarySearch(Queue* queue, int x)` que realiza busca binária pelo valor `x`, assumindo que os elementos da fila estão em ordem crescente. A função deve retornar 1 se `x` for encontrado, ou 0 caso contrário. Sua implementação deve respeitar a complexidade $O(\log_2 n)$.

Questão 3 (2,0 pontos)

Você está desenvolvendo um painel de monitoramento para uma distribuidora de remédios. O painel exibe constantemente qual é o pedido mais urgente a ser processado, com base em um nível de urgência representado por um número inteiro. Novos pedidos são adicionados com frequência, mas remoções ou cancelamentos ocorrem apenas eventualmente. O sistema deve manter a ordem de chegada entre pedidos com o mesmo nível de urgência.

- (a) Com base nas estruturas de dados estudadas em aula, qual você recomendaria para representar os pedidos? Justifique sua escolha, mencionando o Tipo Abstrato de Dados (TAD), a estrutura interna utilizada, e os custos computacionais das seguintes operações: inserção, consulta ao pedido mais urgente e remoção do próximo pedido a ser atendido.
- (b) Sabendo que os níveis de urgência variam de 1 (menos urgente) a 5 (mais urgente), seria possível propor uma estrutura mais eficiente do que a sugerida no item anterior? Em caso afirmativo, descreva essa nova estrutura e como seriam realizadas as operações de inserção, consulta ao pedido mais urgente e remoção do próximo pedido a ser atendido. Caso contrário, justifique por que a estrutura anterior continua sendo a melhor escolha, considerando as características do problema.

Questão 4 (2,5 pontos) Implemente o algoritmo de ordenação por inserção (`insertion sort`) para ordenar, **em ordem crescente**, uma lista duplamente encadeada. A estrutura da lista é dada abaixo:

```
struct Node {
    int data;        // Valor armazenado no nó
    Node* next;      // Ponteiro para o próximo nó na lista
    Node* prev;      // Ponteiro para o nó anterior na lista
};
```

Sua implementação deve reorganizar os ponteiros dos nós existentes para ordenar a lista. Não é permitido alocar novos nós nem alterar os valores armazenados.

- (a) Implemente a função `void insertNode(Node** head, Node* newNode)`, que insere o novo nó `newNode` na posição correta da lista, mantendo-a ordenada em ordem crescente.
- (b) Implemente a função `void insertSort(Node** head)`, que ordena a lista utilizando o algoritmo de ordenação por inserção. Você pode assumir que a função anterior está disponível e corretamente implementada.
- (c) Qual é a complexidade assintótica no pior caso da sua implementação da função `insertSort`? Justifique.

Questão 5 (2.0 pontos) Considere a implementação de uma pilha (`Stack`) utilizando duas filas (`Queue`) baseadas em listas encadeadas.

As operações de enfileiramento (`enqueue`), desenfileiramento (`dequeue`) e é vazio (`isEmpty`) estão definidas pelas seguintes funções fornecidas:

```
void enqueue(Queue* queue, int value);
int dequeue(Queue* queue, int* returnValue); // 0 = Vazio, 1 = OK
int isEmpty(Queue* queue); // 1 = Vazio, 0 = Caso contrário
```

A estrutura das filas é definida como:

```
struct Stack {
    Queue* queue1;
    Queue* queue2;
};
```

Você pode assumir que as filas `queue1` e `queue2` já foram alocadas e inicializadas, e que é permitido trocar diretamente os ponteiros entre elas.

- (a) Implemente as funções `void push(Stack* stack, int value)` e `int pop(Stack* stack, int* returnValue)` para inserir e remover um elemento do topo da pilha, respectivamente.
- (b) Qual é a complexidade assintótica dessas operações na sua implementação? Justifique.
Obs: As operações `enqueue`, `dequeue` e `isEmpty` custam 1.